Phishing URL Detection Using Python

Complete Python Code File — PDF Format
Date: 21 October 2026

1. Main Code File: phishing_url_detector.py

Save the following code as **phishing_url_detector.py**. It extracts URL features, trains a Random Forest model, evaluates it, saves the model, and checks user-entered URLs.

```
"""
Phishing URL Detection using Python
Machine Learning Project
"""

import re
import ipaddress
import pandas as pd
import joblib
from urllib.parse import urlparse
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

SUSPICIOUS_WORDS = [
    "login", "signin", "verify", "secure", "account", "update",
    "confirm", "bank", "paypal", "password", "webscr", "free"
]

SHORTENERS = [
    "bit.ly", "tinyurl.com", "goo.gl", "t.co", "is.gd",
    "shorte.st", "cutt.ly", "rb.gy"
]


def extract_features(url: str) -> dict:
    """Extract useful features from a URL."""
    url = url.strip()
    parsed = urlparse(url)
    domain = parsed.netloc.lower()

    features = {
        "url_length": len(url),
        "hostname_length": len(domain),
        "num_dots": url.count("."),
        "num_hyphens": url.count("-"),
        "num_digits": sum(ch.isdigit() for ch in url),
        "num_special_chars": sum(not ch.isalnum() for ch in url),
        "has_at_symbol": int("@" in url),
        "uses_https": int(parsed.scheme == "https"),
        "has_ip_address": int(is_ip_address(domain)),
        "has_suspicious_word": int(any(word in url.lower() for word in SUSPICIOUS_WORDS)),
        "is_shortened_url": int(any(short in domain for short in SHORTENERS)),
        "has_double_slash_redirect": int("//" in parsed.path),
        "num_subdomains": max(domain.count(".") - 1, 0),
    }

    return features


def is_ip_address(domain: str) -> bool:
    """Check whether the domain is an IP address."""
    try:
        ipaddress.ip_address(domain)
        return True
    except ValueError:
```

```
        return False


def create_sample_dataset() -> pd.DataFrame:
    """Create a small demonstration dataset."""
    urls = [
        "https://www.google.com",
        "https://www.amazon.in",
        "https://github.com",
        "https://www.microsoft.com",
        "https://www.wikipedia.org",
        "http://secure-login@bank-verify.com",
        "http://192.168.1.1/login",
        "http://paypal-account-verify.com",
        "https://bit.ly/free-money",
        "http://update-your-account secure-bank.com",
        "https://www.flipkart.com",
        "http://facebook-login-verify.com",
    ]
    labels = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1]

    records = [extract_features(url) for url in urls]
    df = pd.DataFrame(records)
    df["label"] = labels
    return df


def train_model(dataset_path=None):
    """Train the phishing URL detection model."""
    if dataset_path:
        data = pd.read_csv(dataset_path)
    else:
        data = create_sample_dataset()
        print("Using built-in demonstration dataset.")

    X = data.drop(columns=["label"])
    y = data["label"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    model = RandomForestClassifier(
        n_estimators=200,
        max_depth=None,
        random_state=42,
        n_jobs=-1
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print("\nModel Evaluation")
    print("================")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    joblib.dump(model, "phishing_url_model.pkl")
    print("\nModel saved as phishing_url_model.pkl")

    return model


def predict_url(model, url: str):
    """Predict whether a URL is phishing or legitimate."""
    features = pd.DataFrame([extract_features(url)])
```

```
    prediction = model.predict(features)[0]
    probability = model.predict_proba(features)[0][1]

    result = "Phishing URL" if prediction == 1 else "Legitimate URL"
    print(f"URL: {url}")
    print(f"Prediction: {result}")
    print(f"Phishing probability: {probability:.2f}")
    return prediction


if __name__ == "__main__":
    model = train_model()

    while True:
        url = input("\nEnter a URL to check (or type exit): ").strip()
        if url.lower() == "exit":
            break
        if url:
            predict_url(model, url)
```

2. Optional Streamlit App: app.py

Save this file as **app.py** to create a simple web interface. Run it using **streamlit run app.py**.

```
"""
Streamlit Web Interface for Phishing URL Detection
Run: streamlit run app.py
"""

import streamlit as st
import joblib
from phishing_url_detector import extract_features

model = joblib.load("phishing_url_model.pkl")

st.title("Phishing URL Detection System")
st.write("Enter a URL to check whether it is likely legitimate or phishing.")

url = st.text_input("Enter URL")

if st.button("Check URL"):
    if url:
        features = extract_features(url)
        prediction = model.predict([list(features.values())])[0]
        probability = model.predict_proba([list(features.values())])[0][1]

        if prediction == 1:
            st.error(f"Warning: This URL is likely phishing. Risk: {probability:.2%}")
        else:
            st.success(f"This URL appears legitimate. Phishing risk: {probability:.2%}")
    else:
        st.warning("Please enter a URL.")
```

3. How to Run

1. Install requirements: pip install -r requirements.txt 2. Save the main program as phishing_url_detector.py. 3. Run the program: python phishing_url_detector.py 4. Enter any URL when prompted. 5. Optional web interface: streamlit run app.py

4. Notes

The built-in dataset is only for demonstration. For a college project or real deployment, use a larger labeled dataset containing both legitimate and phishing URLs. The model can then be retrained using

train_model("phishing_dataset.csv").